Generics and Type Erasure

10 min read Updated 28 Jul 2026 Topic 23 of 65

Bookmark Mark done

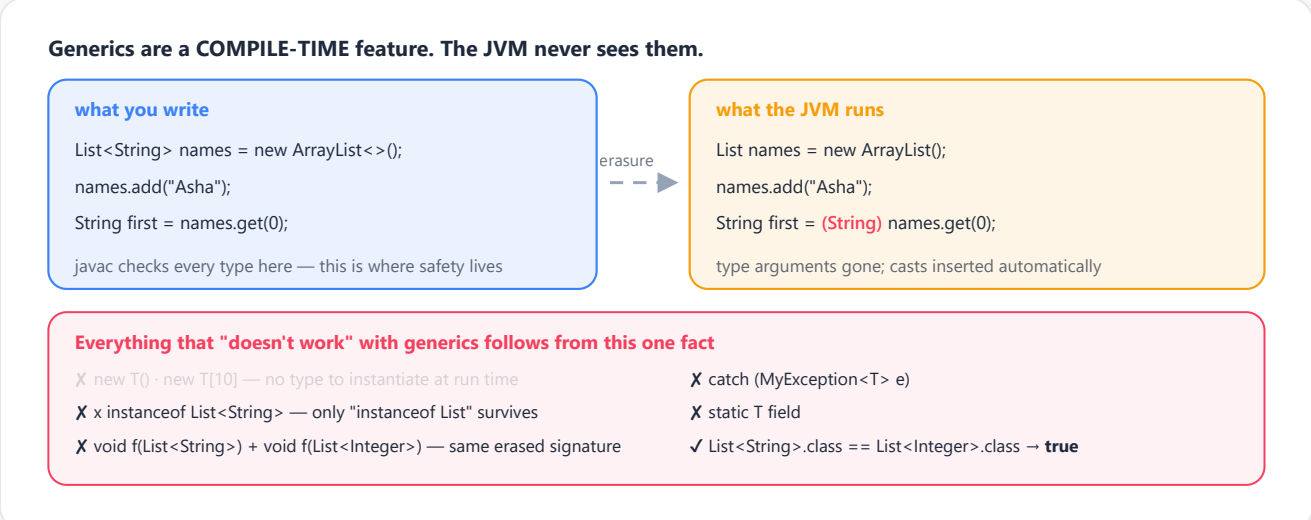
Why it matters

Generics look like a syntax feature and are really a compile-time contract that vanishes before the program runs. Almost every "why can't I do this?" question about generics — no `new T()`, no generic arrays, no overloading on type arguments — has the same one-word answer: **erasure**. Knowing that single mechanism lets you derive the whole list of restrictions instead of memorising it, which is exactly what interviewers are testing.

The concept, simply

Generics let a class or method be written once and used with many types while keeping full type checking. `List<String>` means the compiler will reject anything but a `String` going in, and will hand you a `String` coming out with no cast.

Type erasure is how it is implemented: `javac` checks all the types, then *deletes* the type arguments and inserts casts where needed. The bytecode contains a raw `List`.



Erasure in one picture. The safety is entirely in the compiler; by run time the type argument is gone, which is why the operations at the bottom cannot work.

Why implement it this way? Backward compatibility. Generics arrived in Java 5, ten years into the language's life, and billions of lines of existing code used raw collections. Erasure meant generic code and pre-generic code could interoperate and share the same `.class` format — a `List<String>` could be passed to a Java 1.4 library expecting a `List`. The price is everything below.

How it works

Erasure replaces each type parameter with its **leftmost bound**, or `Object` if it is unbounded:

You write	The JVM sees
`class Box<T>`	`class Box` with `T` → `Object`
`class Box<T extends Number>`	`class Box` with `T` → `Number`
`List<String>`, `List<Integer>`	both just `List`
`<T> void f(T t)`	`void f(Object t)`

Everything that is impossible follows mechanically. You cannot write `new T()` because there is no `T` at run time to instantiate. You cannot write `x instanceof List<String>` because only `List` survives to be tested. You cannot overload `f(List<String>)` and `f(List<Integer>)` because after erasure both are `f(List)` — the same signature. You cannot have a static field of type `T`, because statics belong to the class and the class has no type argument. And `List<String>.class == List<Integer>.class` is `true`, because there is only one `Class` object.

Generic arrays deserve their own note because the reason is subtler. Arrays are *covariant* and *reified* — they know their component type at run time and check every store, throwing `ArrayStoreException`. Generics are *invariant* and erased. Mixing them would let a compile-time-checked write corrupt an array without any run-time check catching it, so `new T[10]` is banned outright.

Code examples

The core payoff — a bug moved from run time to compile time:

JAVA Copy

```java
// Pre-generics: fails at RUN time, and only when that line executes.
List raw = new ArrayList();
raw.add("hello");
raw.add(42);                              // compiles happily
String s = (String) raw.get(1);           // ✗ ClassCastException

// With generics: fails at COMPILE time, at the point of the mistake.
List<String> typed = new ArrayList<>();
typed.add("hello");
// typed.add(42);                         // ✗ will not compile
String s = typed.get(0);                  // no cast needed
```

What erasure forbids, with the reason for each:

JAVA Copy

```java
class Broken<T> {
    // T value = new T();                 ✗ no T at run time to instantiate
    // T[] array = new T[10];             ✗ arrays are reified, generics are
    // static T shared;                   ✗ statics belong to the class, not
    // void f(Object o) {
    //     if (o instanceof T) { }        ✗ nothing to test — T is gone
    // }
}

// The overload that looks fine and is not:
// void process(List<String> l) { }
// void process(List<Integer> l) { }
// ✗ "both methods have same erasure process(List)"

System.out.println(new ArrayList<String>().getClass()
        == new ArrayList<Integer>().getClass());   // true — one Class obj
```

The standard workarounds, which are what a follow-up question is really after:

JAVA Copy

```java
class Factory<T> {
    private final Class<T> type;
    Factory(Class<T> type) { this.type = type; }
    T create() throws Exception {
        return type.getDeclaredConstructor().newInstance();    // ✓ real ty
    }
}
Factory<User> f = new Factory<>(User.class);

// 2. Pass a supplier — cleaner in modern Java, and no reflection.
class Factory2<T> {
    private final Supplier<T> maker;
    Factory2(Supplier<T> maker) { this.maker = maker; }
    T create() { return maker.get(); }
}
Factory2<User> f2 = new Factory2<>(User::new);

// 3. For arrays: use a List, or Array.newInstance with a Class token.
@SuppressWarnings("unchecked")
T[] array = (T[]) Array.newInstance(type, 10);
```

Generic methods, where the type parameter is inferred per call:

JAVA Copy

```java
// The <T> before the return type declares the method's OWN parameter.
public static <T> List<T> firstN(List<T> source, int n) {
    return new ArrayList<>(source.subList(0, Math.min(n, source.size())));
}
List<String> top = firstN(names, 3);        // T inferred as String

// Multiple bounds — class first, then interfaces:
public static <T extends Comparable<T> & Serializable> T max(List<T> list)
    return list.stream().max(Comparator.naturalOrder()).orElseThrow();
}
```

Under the hood

interface declares `compareTo(Object)` — a different signature, so polymorphism would break. The compiler silently generates a synthetic **bridge method** `compareTo(Object)` that casts and delegates to your real one. You can see it with `javap -c`, and it is why an unexpected `ClassCastException` can surface from a line you never wrote.

Not everything is erased. Type arguments that appear in a class or method *signature* are retained as metadata and readable by reflection — which is how Jackson and Hibernate figure out that a field is a `List<User>` and deserialise it correctly. What is erased is the type of an *instance*: a `List<String>` object cannot tell you it is a list of strings. The distinction is worth stating precisely, because "generics are erased" over-simplifies it.

Heap pollution and `@SafeVarargs`. A generic varargs parameter is compiled as an array of the erased type, which reopens the array/generic hole — the method can be handed an array whose actual element type is wrong. That is *heap pollution*, and it is why such methods produce an "unchecked generic array creation" warning. `@SafeVarargs` suppresses it, and you should only apply it when the method merely reads the varargs array and never stores anything into it or lets it escape.

Common mistakes

- **Using raw types.** `List list = new ArrayList();` turns off generic checking for every operation on that reference — including ones that would otherwise be safe.
- **Expecting `List<String>` to be a `List<Object>`.** Generics are invariant. A `List<Object>` would accept an `Integer`, which would corrupt the list.
- **Blanket `@SuppressWarnings("unchecked")`.** Put it on the narrowest possible declaration, with a comment saying why the cast is safe.
- **Trying to overload on type arguments.** Identical after erasure — it will not compile.
- **Creating generic arrays.** Use a `List`, or `Array.newInstance` with a class token.
- **Assuming reflection can see an instance's type argument.** Field and method signatures keep it; objects do not.
- **`@SafeVarargs` on a method that stores into the array.** That is exactly the unsafe case the warning exists for.

Q: "What is type erasure?"

A: The compiler checks all generic types and then removes the type arguments, replacing each type parameter with its leftmost bound — or `Object` if unbounded — and inserting casts where needed. So `List<String>` and `List<Integer>` are both plain `List` in bytecode and share one `Class` object. It was done for backward compatibility, so generic and pre-generic code could interoperate when generics arrived in Java 5.

Q: "Why can't you write `new T()`?"

A: Because `T` does not exist at run time — erasure removed it, so there is no class to instantiate. The workarounds are to pass a `Class<T>` token and use reflection, or, more cleanly, to pass a `Supplier<T>` such as `User::new`.

Q: "Why are generic arrays not allowed?"

A: Arrays and generics have incompatible type systems. Arrays are reified and covariant — they carry their component type at run time and throw `ArrayStoreException` on a bad store. Generics are erased and invariant. Allowing `new T[10]` would let a compile-time-checked write corrupt an array with no run-time check to catch it, so the language bans it and you use a `List` instead.

Q: "What is a bridge method?"

A: A synthetic method the compiler generates to make erasure work with polymorphism. If you implement `Comparable<Employee>` you write `compareTo(Employee)`, but the erased interface declares `compareTo(Object)`. The compiler adds a bridge `compareTo(Object)` that casts and delegates to yours, so dynamic dispatch still finds the right method. It shows up in `javap -c` output and in occasional stack traces.

A: No — generics are invariant, deliberately. If it were allowed you could assign a `List<String>` to a `List<Object>` reference and then add an `Integer`, breaking the original list with no error. Wildcards exist to recover the flexibility safely: `List<? extends Object>` can read, `List<? super String>` can write.

Q: "Can reflection see generic types?"

A: Partly, and the distinction matters. Type arguments in a field, method or class *signature* are kept as metadata — that is how Jackson knows a field is `List<User>`. What is not available is the type argument of an *instance*: given a `List` object, you cannot ask what it contains. So "generics are erased" is true of objects, not of declarations.

Q: "What is heap pollution?"

A: When a variable of a parameterised type refers to an object that is not of that type — most commonly through generic varargs, which compile to an array of the erased type and so reopen the array/generics hole. It usually surfaces as a `ClassCastException` far from the cause. `@SafeVarargs` silences the warning and should only be used when the method just reads the varargs array and never writes to it or lets it escape.

Cheat-card

- **Erasure**: type args checked, then deleted; `T` → leftmost bound or `Object`; casts inserted.
- Reason: **backward compatibility** with pre-Java-5 code.
- Forbidden by erasure: `new T()` · `new T[]` · `instanceof T` · static `T` · overload on type args · generic `catch`.
- `List<String>.class == List<Integer>.class` → **true**.
- Workarounds: `Class<T>` **token** or `Supplier<T>`.

- Signatures keep type args (reflection works); **instances do not**.
- Never use **raw types** — they disable checking wholesale.

QUICK CHECK — no marks, just for you.

1. Why can you not write 'new T()' inside a generic class?

- ◯ Because generics are only allowed on interfaces
- ◯ Because the JVM forbids reflection in generic code
- ◯ Because T might not have a no-arg constructor
- ◯ Because erasure removes the type argument, so there is no T at run time to instantiate

Check answer

2. What does new ArrayList<String>().getClass() == new ArrayList<Integer>().getClass() evaluate to?

- ◯ true — after erasure both are plain ArrayList, sharing one Class object
- ◯ false — each parameterisation gets its own Class
- ◯ It does not compile
- ◯ It depends on whether the lists contain elements

Check answer

3. What is a bridge method?

A method generated by the JIT to inline across class boundaries

- A synthetic method the compiler generates so erasure still works with polymorphism — for example a compareTo(Object) that casts and delegates to your compareTo(Employee)
- A method that converts between primitives and wrappers
- A default method that connects two interfaces

Check answer

Previous
Fail-Fast vs Weakly Consistent Iterators

Next
Bounded Types and Wildcards (PECS)

LEARN MORE

ARTICLES & DOCS

- Type erasure — The Java Tutorials
  docs.oracle.com
- Types, values and variables (JLS §4)
  docs.oracle.com
- Project Valhalla
  openjdk.org

VIDEOS



What is Type Erasure in Generics | How it Works | Java
CoedMaster



Java Generics Finally Explained (Wildcards, PECS & More) | Geekific
Geekific

Hand-picked references for this lesson.



Java Interview Kit